35 PUT VS PATCH — WHAT THE DIFFERENCE ACTUALLY MEANS, AND WHY THIS APP HAS BOTH

student.controller.ts now has two routes pointing at the exact same path AND the exact same service call:

```
@Put('updatestudent/:id')
update(@Param('id') id: string, @Body() dto: UpdateStudentDto) { ... }

@Patch('updatestudent/:id')
patch(@Param('id') id: string, @Body() dto: UpdateStudentDto) { ... }
```

Since they call identical code, the difference between them isn't behavioral in this codebase — it's about HTTP semantics: what each verb PROMISES the client and any tooling (browsers, proxies, API gateways, caches) sitting in between.

THE ACTUAL DEFINITION

PUT = "replace the ENTIRE resource with exactly this representation."

```
A fully-compliant PUT client sends the WHOLE object — every
field — because anything omitted is technically supposed to
be treated as cleared/reset, not left alone.
```

PATCH = "apply a PARTIAL modification to the resource."

```
You send only the fields you want to change; everything else
on the document is left untouched.
```

This is a REST convention (from the HTTP spec, RFC 5789 for PATCH), not something Nest or Mongoose enforces for you — nothing stops you from sending a partial body to a PUT route, which is exactly what this app does today.

WHY THIS APP'S "PUT" WAS ALREADY BEHAVING LIKE PATCH

Look at update-student.dto.ts:

```
export class UpdateStudentDto extends PartialType(CreateStudentDto) {}
```

`PartialType` makes name/age/email/course ALL OPTIONAL. And the service method backing both routes:

```
async update(id: string, dto: UpdateStudentDto) {
  const updatedStudent = await this.studentModel
    .findByIdAndUpdate(id, dto, { new: true })
    .exec();
  ...
}
```

`findByIdAndUpdate` only touches the fields present in `dto` — it merges them into the existing document rather than replacing the whole thing. So even before the PATCH route was added, hitting PUT with just `{ "age": 21 }` would NOT wipe out name/email/course. That merge behavior is PATCH semantics, regardless of which HTTP verb the route was labeled with.

SO WHY ADD PATCH AT ALL, IF THE CODE BEHAVES THE SAME EITHER WAY?

Because the VERB is documentation for everyone outside your own function body — other developers, API clients, tools that auto-generate docs from routes (Swagger/OpenAPI), and HTTP-aware infrastructure. Exposing `PATCH /student/updatestudent/:id` tells a caller "partial update is explicitly supported and expected here" without them needing to read update.dto.ts to discover that PUT was secretly lenient. Keeping PUT around too means existing clients that already call PUT don't break — both routes now honestly describe the one merge-style update() behavior underneath.

THE RULE OF THUMB, IF YOU WANT TO BE STRICT ABOUT IT LATER

```
Want a route that truly matches PUT's contract (whole-resource
replace, missing fields get cleared)?
    -> use a DTO based on CreateStudentDto (all fields required),
       and swap `findByIdAndUpdate(id, dto, { new: true })` for
       something like `findOneAndReplace()`.
```

> Want a route that truly matches PATCH's contract (merge only what's sent)? → exactly what's here already: PartialType DTO + findByIdAndUpdate's default merge behavior.

This app currently only implements the PATCH-style contract, and exposes it under both verb names. That's a reasonable, common simplification for a small CRUD API — just know that a strictly spec-compliant PUT client sending a partial body would be surprised that untouched fields *didn't* get cleared.

See also: **theory/33-async-await-promises.txt** (findByIdAndUpdate is a DB round-trip), student.controller.ts, student.service.ts, student/dto/update-student.dto.ts